

Corrigibility for Learned Systems: Extending Structural Accountability to AI

Working Draft

February 2026

Abstract

This paper extends the corrigibility framework for Digital Public Infrastructure (DPI) to learned systems—AI models whose behavior is determined by training data rather than explicit code. While the companion paper establishes five structural tests (EXIT, CODE, AUDIT, GOVERN, FORK) for infrastructure accountability, learned systems present unique challenges: stochastic outputs, opaque training pipelines, compute-gated reproduction, and temporal variety gaps. We demonstrate that the five tests remain invariant for AI systems; only the verification methods change. The paper introduces three contributions: (1) the **LWD-R disclosure requirement** extending CODE to training pipelines, (2) the **Temporal Variety Gap** explaining why AI corrigibility requires ongoing governance rather than deployment-time verification, and (3) **Compute Capture** as a resource barrier to FORK that transforms “open weights” into performative transparency. Analysis of agentic scaling reveals that incorrigible infrastructure becomes structurally incompatible with AI automation, producing Governance Denial of Service (GDoS). The framework provides falsifiable tests for AI infrastructure accountability, distinguishing structural corrigibility from operator-focused “AI alignment.”

Keywords: AI Governance, Corrigibility, Learned Systems, Compute Capture, Agentic Economy, LWD-R, Open Source AI

1 Introduction

The companion paper [Author, 2026] establishes a corrigibility framework for Digital Public Infrastructure, deriving five structural tests from cybernetics, commons governance, and free software:

- **EXIT:** Reversibility of participation
- **CODE:** Inspectability of logic
- **AUDIT:** Independent verification
- **GOVERN:** Binding constraint
- **FORK:** Capacity to reproduce

This paper stress-tests the framework against learned systems—infrastructure where behavior is determined by parameters learned from data (AI/ML) rather than explicit code logic. If the five tests are truly fundamental, they must apply not only to deterministic infrastructure but also to stochastic systems where traditional verification assumptions collapse.

The Challenge. Learned systems dismantle the assumption that source code is the sole arbiter of behavior. In AI, identical inputs may yield stochastic outputs; behavior is defined by training data; and opacity is often inherent rather than contrived. Yet the collapse of deterministic transparency does not invalidate corrigibility’s structural necessity. As systems become less intelligible, external correction becomes *more* critical.

Thesis. The five corrigibility tests remain invariant for learned systems. What changes is the verification method and the resource barriers to compliance.

Scope. This paper addresses AI systems deployed as infrastructure—models that determine citizen outcomes at scale. It does not address consumer AI products where exit is trivial, nor research systems without deployment impact. The focus is on AI that functions as DPI: identity verification models, welfare eligibility systems, public-facing chatbots with administrative authority, and autonomous agents operating within mandatory infrastructure.

Paper Structure. Section 2 establishes that the five tests remain invariant. Section 3 extends the CODE test via the LWD-R disclosure requirement. Section 4 introduces the Temporal Variety Gap. Section 5 analyzes resource barriers to FORK, including compute capture. Section 6 examines agentic scaling and Governance Denial of Service. Section 7 addresses workflow sovereignty and the Rule of Workflow. Section 8 provides architecture-specific risk analysis. Section 10 discusses implications and limitations.

2 The Invariance of the Standard

Proposition 2.1. The five corrigibility tests remain invariant for learned systems. What changes is the verification method.

The structural requirement for corrigibility derives from Ashby’s Law of Requisite Variety: a controller must have at least as many response states as the system it governs [Ashby, 1956]. This requirement is independent of whether the controlled system is deterministic or stochastic. An AI model that affects citizen outcomes at population scale generates the same variety of potential harms as deterministic infrastructure; therefore, the governance apparatus must possess equivalent corrective variety.

The table below illustrates how the mechanical verification of each test adapts to the probabilistic nature of AI without softening the structural requirement.

Test	Deterministic Verification	Learned System Verification
EXIT	Can refuse the system	Disclosure + non-AI alternative
CODE	Source code determines behavior	Layered transparency across training pipeline
AUDIT	Inspect logic and outputs	Statistical bounds + continuous monitoring
GOVERN	Maintainer and RFC process	Governance over objectives and value tradeoffs
FORK	Copy code and deploy	Resource forkability (compute + data)

Table 1: Verification methods for deterministic vs. learned systems

Key Insight. The shift from deterministic to learned systems does not weaken the tests; it *strengthens* the case for structural corrigibility. When behavior becomes less predictable, the capacity for external correction becomes more essential. A deterministic system that fails silently can be debugged; a learned system that fails silently accumulates error without detection.

3 What Is “Source” in a Learned System?

A central challenge in complying with the **CODE** test is redefining the object of inspection. In a learned system, behavior is distributed across a lineage of distinct artifacts. The inference code is merely the execution engine; the actual behavioral logic is encoded in the weights, which are in turn downstream of the training corpus, filtering decisions, and RLHF preferences.

Principle 3.1 (Source Distribution). Publishing inference code alone does not satisfy the CODE test. It allows an auditor to run the machine, but not to understand why it produces specific outputs.

To bridge this gap, compliance requires structured disclosure across the training pipeline, such as **Data Sheets for Datasets** [Gebru et al., 2021] and **Model Cards** [Mitchell et al., 2019]. These artifacts function as the “source code” for structural evaluation; withholding them renders the system a black box.

3.1 The LWD-R Disclosure Requirement

For world models and agentic systems, the traditional transparency requirements (code, weights, data) are necessary but insufficient. We extend the disclosure requirement to the **LWD-R Triad**:

Definition 3.1 (LWD-R). A learned system satisfies LWD-R disclosure if the following artifacts are publicly available:

- **L (Logic)**: Model architecture and inference code
- **W (Weights)**: Trained parameters
- **D (Data)**: Training corpus with provenance (Data Sheets)
- **R (Representation Schema)**: The ontological categories the model uses to parse reality

The fourth component, **R**, is critical for world models that encode a de facto administrative ontology. When an AI system classifies citizens, transactions, or risks, it instantiates categories that become infrastructural facts. Failure to disclose **R** renders the model’s worldview non-inspectable.

3.2 Ontological Opacity and Representation Capture Risk

We define **Representation Capture Risk (RCR)** as the condition where a society becomes dependent on a model’s categorical schema without structural means to contest or modify those categories.

Definition 3.2 (Representation Capture Risk). Let ΔO represent the **ontological contestability** of a system, defined as the degree to which affected parties can challenge, modify, or replace the representation schema. RCR occurs when:

$$\Delta O \rightarrow 0 \tag{1}$$

Under RCR, the model’s internal categories become non-contestable administrative facts. A welfare eligibility model that classifies applicants into risk tiers, or a world model that parses geopolitical actors into threat categories, exercises definitional power over reality. When $\Delta O = 0$, the system fails CODE not because the weights are hidden, but because the *meaning structure* is opaque.

Extension to CODE. This extends the CODE test for world models: compliance requires not merely weight publication but **representation schema disclosure**—machine-readable specifications of the ontological categories the model employs, with documented justification for category boundaries.

4 The Temporal Variety Gap

A central theoretical contribution of this framework is the identification of the **Temporal Variety Gap**: a fundamental challenge that distinguishes learned systems from traditional infrastructure and explains why AI corrigibility requires ongoing governance, not merely deployment-time verification.

Unlike traditional software, which remains static until explicitly patched, learned systems possess a distinct vulnerability related to time. The variety of the controller (V_c) is frozen at the moment of training parameterization (θ), while the variety of the world it operates upon (V_d) continues to expand.

$$\Delta(t) = V(\text{disturbance}; t) - V(\text{controller}; \theta) \quad (2)$$

Without mechanisms for retraining or fine-tuning accessible to the community, the variety gap $\Delta(t)$ widens over time. Thus, AI corrigibility is not a deployment property, but a *persistence condition*.

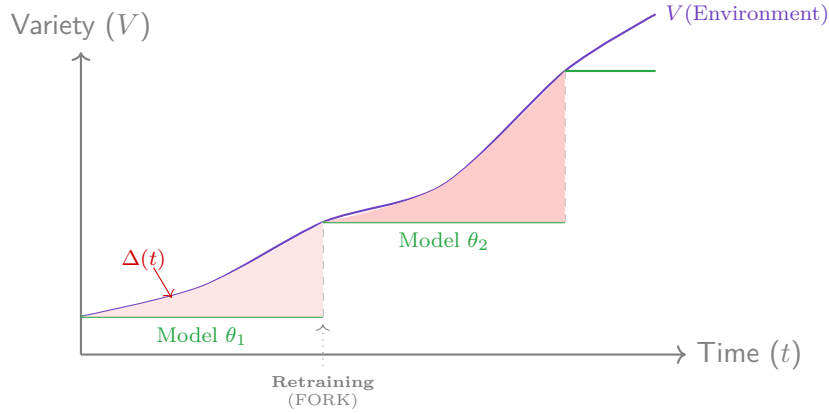


Figure 1: **The Temporal Variety Gap.** Environmental variety $V(e)$ evolves continuously (curve). A fixed model θ (stepped line) diverges from reality. The shaded area represents accumulated error. Only **FORK** rights allow the gap to be closed via retraining.

Claim 4.1. A learned system that passes all five corrigibility tests at deployment may fail them over time if the community lacks the resources to retrain. Corrigibility for AI is a dynamic property, not a static certification.

5 Resource Barriers to Forkability

The right to **FORK** a learned system is constrained by physical resources rather than just copyright. In traditional software, the cost to fork is the cost of storage; in AI, the cost to fork is the cost of compute.

Layer	Cost	Barrier
Inference code	Minimal	None
Running open weights	Low	Minor
Fine-tuning	Moderate	Economic
Retraining from scratch	Very high	Structural

Table 2: Resource barriers to forking learned systems

Claim 5.1. Legal permission to fork without access to compute or data is meaningless. This is an infrastructure question, not a licensing question.

5.1 The Compute Capture Problem

Principle 5.1 (Compute Capture). Training forkability is functionally impossible when the compute cost to retrain a model exceeds the compute accessible to plausible non-operator actors by more than a policy-determined multiplier. Releasing weights while hoarding compute is **inference forkability without training forkability**.

If a company releases weights but training cost exceeds $100\times$ global academic compute capacity, training forkability is zero.

Formal Definition. Let C_{train} be the compute cost required to retrain a model, and $C_{\text{accessible}}$ be the compute accessible to plausible non-operator actors (academic clusters, government research consortiums, distributed compute pools). Training forkability is functionally impossible when:

$$C_{\text{train}} > \kappa \cdot C_{\text{accessible}} \quad (3)$$

where κ is an accessibility multiplier (policy constant, typically $1 \leq \kappa \leq 2$).

The formal FORK determination for training becomes:

$$\text{FORK}_{\text{training}} = \mathbf{1}[C_{\text{train}} \leq \kappa \cdot C_{\text{accessible}}] \wedge \mathbf{1}[\text{data_manifest_available}] \quad (4)$$

where $\mathbf{1}[\cdot]$ is the indicator function.

5.2 Compute as Governance Infrastructure

This analysis implies that **compute is itself DPI**. Legal forkability is insufficient absent practical compute accessibility. Architectural responses include:

1. **Public Compute Nodes:** State-sponsored clusters with multi-stakeholder governance
2. **Mandatory Cost Disclosure:** FLOP estimates and data manifests
3. **Federated Training Pools:** Distributed training across independent nodes
4. **Compute Vouchers:** Grants for reproduction attempts

5.3 Minimum Evidence for Learned System Forkability

To prevent “FORK fails” from becoming a rhetorical claim rather than a verifiable determination, learned systems must satisfy specific evidentiary requirements:

Forkability Determination. A learned system passes FORK if:

1. All “Required” artifacts are publicly available under permissive terms
2. The compute cost estimate demonstrates feasibility for at least one non-operator entity
3. Either a successful third-party reproduction exists, OR the training code + data manifest enables reproduction in principle

5.4 Inference vs. Training Forkability

Systems that publish weights but withhold training data or code achieve only **inference forkability**, not **training forkability**. Inference forkability permits running the model; training forkability permits correcting it. Only the latter satisfies FORK for governance purposes.

Table 3: Evidence requirements for FORK in learned systems

Artifact	Status	Verification
Inference code	Required	Open source license; build reproducibility
Model weights	Required	Public download; hash verification
Training data manifest	Required	Data Sheets [Gebru et al., 2021] with provenance
Training checkpoint	Recommended	Enables fine-tuning without full re-train
Training code	Required	Includes preprocessing, hyperparameters
Compute cost estimate	Required	FLOP estimate or cloud cost documentation
Successful reproduction	Decisive	Third-party retrain within 10% of benchmark

Concrete Examples. As of 2026, most “open” models achieve only inference forkability:

- **Inference forkability only:** Llama 3 (Meta), DeepSeek, Gemma (Google), Mistral. Weights are published, but training data remains proprietary and full training code is not disclosed. Users can run and fine-tune these models but cannot independently reproduce or audit the training process.
- **Training forkability:** OLMo (Allen Institute), Pythia (EleutherAI). These projects publish weights, complete training code, and full data manifests (Dolma and The Pile respectively). Third parties have successfully reproduced training runs, satisfying the FORK test.

The distinction matters: when Llama or Gemma exhibit unexpected behavior, the community can observe symptoms but cannot diagnose root causes in training data. When OLMo exhibits unexpected behavior, researchers can trace the issue to specific training examples. **Open weights with closed training pipelines is inference transparency without correction capacity.**

Alignment with OSI Definition. The Open Source Initiative’s Open Source AI Definition 1.0 (2024) codifies this distinction, requiring: (1) data information sufficient for recreation, (2) complete training code, and (3) model parameters. By this standard, Llama, Gemma, DeepSeek, and Mistral do not qualify as open source AI despite marketing claims. This framework’s FORK test operationalizes the same principle: without training forkability, governance correction is structurally impossible.

6 Agentic Scaling and Governance Denial of Service

As DPI transitions to support the “Agentic Economy,” corrigibility moves from a moral preference to an engineering requirement. Autonomous agents lack the biological resilience to handle bureaucracy.

Principle 6.1 (Structural vs. Alignment Distinction). This framework distinguishes structural corrigibility from “AI Alignment.” Alignment asks: *Can the operator control the system?* Corrigibility asks: *Can the affected subject correct the system?* Structural legitimacy requires the latter.

6.1 Corrigibility as an API Contract for Agents

- **EXIT is Exception Handling:** To an agent, a mandatory system with no exit path appears as a logic deadlock.
- **CODE is Documentation:** Agents require inspectability of logic to form valid plans.
- **AUDIT is Observability:** An optimization agent cannot function without a reliable failure signal.
- **GOVERN is Constraint Discovery:** Advisory guidelines result in undefined behavior for software agents.

Claim 6.1. Incurrigible infrastructure is structurally incompatible with AI automation.

6.2 The Human Friction Buffer

Incurrigible infrastructures may appear stable under human load because humans absorb friction through delay, compliance, and informal workaround. Autonomous agents remove this buffering layer.

Illustrative Scenario. Consider an autonomous procurement agent that interacts with a mandatory payment API. On encountering a compliance flag, the agent receives an opaque error code. No machine-readable error taxonomy exists, and dispute resolution is human-mediated. The agent retries, triggering repeated failures and queue saturation. Human agents absorb such friction via manual override and phone calls; fleets of autonomous agents cannot. The result is a governance denial-of-service (GDoS) that cascades across the payment rail.

6.3 Governance Denial of Service (GDoS)

As agentic interaction scales, centralized governance mechanisms face **Governance Denial of Service (GDoS)**: the volume, speed, and strategic diversity of agent actions exceeds the system’s capacity to audit, interpret, and correct behavior.

This is not a security failure but a control-theoretic inevitability. Systems lacking distributed audit and rapid, binding governance lack the requisite variety to remain stable under agentic scaling. The correction velocity inequality predicts precisely this failure mode: when $\frac{dE}{dt} \gg \frac{dC}{dt}$, the system diverges. Agents accelerate $E(t)$; centralized governance cannot accelerate $C(t)$.

Claim 6.2. Agentic scaling amplifies the correction velocity inequality. Systems that are marginally stable under human interaction rates become catastrophically unstable under agent interaction rates. The human friction buffer is not a feature; it is a hidden subsidy that masks architectural incurrigibility.

6.4 Agentic Interoperability Requirements

For an autonomous agent $\mathcal{A}$ to operate reliably within infrastructure $\mathcal{I}$, the following predicates must be strictly True:

1. **Reversibility:** $\forall$ state $s_t \in \mathcal{S}$, $\exists$ transition s_{t+1} (via **EXIT**) such that $\text{Cost}(s_{t+1}) < \infty$. (No deadlocks).
2. **Decidability:** $\forall$ input x , $\text{Compute}(x) \rightarrow \{0,1\}$ is inspectable via **CODE**. (No hidden constraints).

Failure of these predicates renders $\mathcal{I}$ a “Hazmat Environment” for automated agents, effectively blocking the agentic economy.

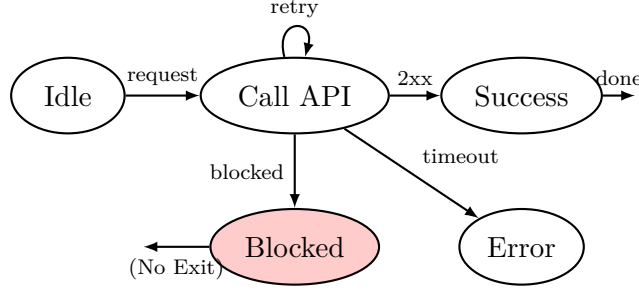


Figure 2: **Agentic automaton.** Without EXIT and observable error codes, agent loops indefinitely, times out, or escalates incorrectly.

6.5 Agentic Skills as Corrigible Primitives

As infrastructure transitions to the Rule of the Workflow, the fundamental unit of governance can no longer be the static database record, nor can it be the stochastic model weight. To subject autonomous systems to democratic constraint, the architecture must adopt a new structural primitive: the **Agentic Skill**.

Definition 6.1 (Agentic Skill). An Agentic Skill is a bounded, machine-readable orchestration protocol that encapsulates stochastic inference within strict preconditions, deterministic tool execution, and explicit termination criteria. It acts as the technical instantiation of the Agentic API Contract.

6.5.1 Bounding Stochasticity

In learned systems, behavior is probabilistic. However, public infrastructure requires deterministic accountability. The Agentic Skills architecture resolves this tension by separating the *reasoning engine* (the stochastic model) from the *action boundary* (the deterministic skill definition).

Principle 6.2 (Action Boundary Governance). If the state cannot govern the neural network’s weights, it must govern the action boundary. Agentic Skills provide the deterministic envelope that wraps around probabilistic inference.

When an agent attempts a state transition, the workflow orchestration does not rely on the model’s internal alignment. Instead, it invokes a defined Skill that imposes exogenous constraints:

1. **Exception Handling (EXIT)**: The skill definition dictates explicit fallback paths and termination triggers, preventing the infinite loops that cause Governance Denial of Service (GDoS).
2. **Ontological Disclosure (CODE)**: By encoding the agent’s workflow into declarative, human-readable files (e.g., YAML/Markdown skill definitions), the system’s operational ontology becomes public. This satisfies the Representation (R) requirement of the LWD-R triad.
3. **Telemetry (AUDIT)**: Skills enforce structured inputs and outputs, transforming opaque “conversations” into discrete, auditable transaction logs.
4. **Binding Rules (GOVERN)**: The execution of the skill acts as the non-overrideable controller. An LLM cannot prompt-inject its way out of a hard-coded skill constraint.

6.5.2 Defeating Workflow Capture via Open Skills

The critical danger of agentic DPI is Epistemic Delegation: the loss of state sovereignty to proprietary orchestration layers. We previously defined the Workflow Capture Coefficient (WCC)

as a function of reproduction cost, ontological substitutability, and portability.

Standardized Agentic Skills fundamentally disrupt this capture vector. If a state defines its administrative processes using open skill protocols rather than vendor-locked pipelines:

- $C_{\text{workflow}} \rightarrow 0$: The orchestration logic is transparent and easily replicable
- $O_{\text{substitutability}} \rightarrow 1$: Administrative categories are defined in open skill schemas, not hidden in a vendor’s latent space
- $M_{\text{portability}} \rightarrow 1$: The state can swap the underlying LLM without losing or breaking the workflow logic

Consequently, adopting an open Agentic Skills architecture drives the WCC toward zero, preserving the state’s interpretive sovereignty and satisfying the **FORK** test for the agentic era.

Claim 6.3. You don’t govern the AI; you govern the Skills the AI is allowed to use. This is the architectural translation of corrigibility into the agentic domain.

7 Workflow Sovereignty and the Rule of Workflow

The final mutation of digital infrastructure is the transition from static registries to **Workflow Orchestration** layers (e.g., Palantir-style data fusion or agentic AI pipelines). In this regime, the operating layer of the state is no longer a ledger of records, but an orchestration of decisions.

7.1 From Rule of Law to Rule of Workflow

In classical governance, policy is defined in statute, executed by bureaucracy, and reviewed by courts. In a “Rule of Workflow” regime, policy becomes emergent from the orchestration logic: data schemas define reality, prompts define interpretation, and API contracts define valid inputs.

Workflow Capture is formalized as follows. Let W be the orchestration layer, M the underlying model, S the state policy function, and D the decision output. While classical governance assumes $S \rightarrow D$, AI-native governance operates as:

$$S \rightarrow W(M, \text{Data}) \rightarrow D \quad (5)$$

If W is proprietary and non-forkable, the state’s ability to alter policy (ΔS) vanishes because outcomes are constrained by the workflow’s technical affordances. Policy debate is reduced to parameter tuning within a vendor-controlled ontology.

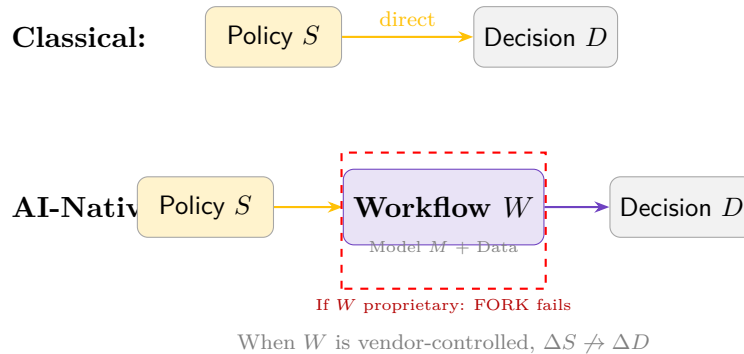


Figure 3: **Workflow Capture.** Classical governance: policy directly determines decisions. AI-native governance: policy is mediated by workflow orchestration. If the workflow is proprietary, policy changes cannot reliably change outcomes.

7.2 The Workflow Capture Coefficient (WCC)

To measure the risk of **Epistemic Delegation**, the framework proposes the **Workflow Capture Coefficient (WCC)**. When a state adopts proprietary orchestration layers (like Palantir or agentic AI pipelines), it suffers epistemic delegation: the state loses the ability to define reality because policy becomes constrained by the vendor’s technical affordances.

7.2.1 The Three Variables of Capture

WCC is defined as a function of three normalized variables (each in $[0, 1]$):

- C_{workflow} : The normalized cost of reproducing the orchestration stack (0 = trivial, 1 = infeasible)
- $O_{\text{substitutability}}$: The ease of altering the underlying ontological schema (0 = locked, 1 = fully substitutable)
- $M_{\text{portability}}$: The data and model portability across implementations (0 = proprietary lock-in, 1 = fully portable)

7.2.2 The Harmonic Mean Formulation

The WCC is computed using a modified weighted harmonic mean:

$$\text{WCC} = 1 - \frac{3}{\frac{1}{1-C_{\text{workflow}}} + \frac{1}{O_{\text{substitutability}}} + \frac{1}{M_{\text{portability}}}} \quad (6)$$

Why Harmonic Mean? The choice of harmonic mean rather than arithmetic mean provides the mathematical “teeth” of this formalization. The harmonic mean has a critical property: it is *heavily dominated by its smallest input*. If any single dimension approaches zero (e.g., if portability is near-zero, even if code is open and substitutable), the harmonic mean collapses, driving WCC toward 1 (high capture).

This directly operationalizes cybernetic theory: a feedback loop fails if *any* of its components fail. An arithmetic average would allow a high score in one dimension to hide complete failure in another. The harmonic mean mathematically prevents this compensation.

7.2.3 Worked Examples

Aadhaar (High Capture).

- $C_{\text{workflow}} = 0.95$ (requires replicating UIDAI’s entire biometric infrastructure)
- $O_{\text{substitutability}} = 0.1$ (biometric templates are proprietary and non-interoperable)
- $M_{\text{portability}} = 0.05$ (no data export, no credential portability)

$$\text{WCC}_{\text{Aadhaar}} = 1 - \frac{3}{\frac{1}{0.05} + \frac{1}{0.1} + \frac{1}{0.05}} = 1 - \frac{3}{20 + 10 + 20} = 1 - 0.06 = \mathbf{0.94} \quad (7)$$

X-Road (Low Capture).

- $C_{\text{workflow}} = 0.3$ (open-source, documented)
- $O_{\text{substitutability}} = 0.8$ (standard protocols)
- $M_{\text{portability}} = 0.7$ (open data exchange formats)

$$\text{WCC}_{\text{X-Road}} = 1 - \frac{3}{\frac{1}{0.7} + \frac{1}{0.8} + \frac{1}{0.7}} = 1 - \frac{3}{1.43 + 1.25 + 1.43} = 1 - 0.73 = \mathbf{0.27} \quad (8)$$

Interpretation. Aadhaar’s $WCC = 0.94$ indicates near-total workflow capture; X-Road’s $WCC = 0.27$ indicates low capture risk. A high WCC (above threshold ≈ 0.7) triggers FORK failure regardless of whether underlying model weights are open. **Open weights with closed workflows is performative transparency.**

Claim 7.1. If a state procures a system with high WCC, it is not merely buying software; it is mathematically surrendering interpretive sovereignty to a vendor.

7.3 Fallout Governance

When AI decision throughput outruns human legislative correction velocity, the state enters **Fallout Governance**: policy ceases to guide behavior in advance; it instead reacts post-hoc to scandal, litigation, or failure clusters. Governance becomes damage containment: the Collingridge Dilemma at civilizational scale.

8 Architecture-Specific Risk Analysis

The preceding analysis enables architecture-specific risk diagnosis. Different model architectures exhibit distinct failure profiles across the five tests and the RCR dimension.

Table 4: **Topology-Aware Corrigibility Risk.** Impact of architectural class on structural tests. Notation: W = weights opacity, G = gating opacity, R = representation opacity, S = scale barrier, O = ontological capture, F = fragmentation, κ = compute capture.

Architecture	EXIT	CODE	ΔO (RCR)	AUDIT	GOVERN	FORK
Dense LLM	Partial	Fail (W)	Medium	Partial	Fail (S)	Fail (κ)
Mixture of Experts	Medium	Fail (G)	Medium	Fail (S)	Partial	Pass
World Model	Fail (O)	Fail (R)	Very Low	Partial	Partial	Fail (κ)
Edge / SLM	Pass	Pass	High	Fail (F)	Fail (F)	Pass

Interpretation.

- **Dense LLMs** fail FORK due to compute capture (κ). Training costs exceed accessible compute by orders of magnitude.
- **World models** present the highest RCR (lowest ΔO) because their representation schemas encode administrative ontologies that resist contestation.
- **Edge/SLM models** pass EXIT and FORK but may fail GOVERN due to fragmented deployment making coordinated governance infeasible.
- **Mixture of Experts** architectures may pass FORK if expert routing is documented, but gating opacity often violates CODE.

9 Anticipated Objections

9.1 Objection: Deterministic Tests Cannot Apply to Stochastic Systems

A natural critique holds that applying rigid, deterministic structural tests to systems that are inherently stochastic constitutes a category error. How can tests designed for inspectable code apply to systems where identical inputs yield probabilistic outputs and “logic” is encoded in billions of weights?

Response: Verification Methods Evolve, Standards Remain. The framework acknowledges that learned systems dismantle the assumption that source code is the sole arbiter of behavior. However, this does not invalidate structural corrigibility—it *strengthens* its necessity. As systems become less intelligible, external correction mechanisms become more critical, not less.

The key insight is that the **standard remains invariant** while the **verification method adapts**:

- **CODE** shifts from reading source code to requiring the **LWD-R Triad**: Logic (architecture), Weights (parameters), Data (provenance), and Representation (ontological categories).
- **AUDIT** shifts from discrete logic inspection to statistical bounds and continuous monitoring.
- **FORK** shifts from file copying to resource forkability—measured by access to compute and training data, not just open weights.
- **GOVERN** requires deterministic guardrails *around* the stochastic model, governing objectives and value tradeoffs at the infrastructure layer.

Deterministic Boundaries as Engineering Requirement. The framework argues that deterministic structural boundaries are not incompatible with AI but are an *engineering requirement* for the agentic economy. Autonomous agents lack biological resilience for ambiguity. To an AI agent, the five tests function as a necessary API contract:

- EXIT = Exception handling (preventing logic deadlocks)
- CODE = Documentation (enabling valid plan formation)
- AUDIT = Observability (providing reliable failure signals)
- GOVERN = Constraint discovery (since “advisory” guidelines produce undefined behavior)

The Alternative is Surrender. If we accept that “AI is too stochastic to be evaluated by structural rules,” we mathematically surrender to black-box monopolies. The framework does not attempt to inspect the probabilistic “thoughts” of AI models. Instead, it demands structural control over the *pipeline that builds them* (data, compute, representation) and the *infrastructure that deploys them* (workflow orchestrations and guardrails).

9.2 Objection: Compute Capture Makes FORK Trivially Impossible

If frontier model training costs exceed global academic compute capacity by orders of magnitude, doesn’t the framework simply conclude “all frontier AI fails FORK”—a trivially true and useless finding?

Response: The Distinction Is Training vs. Inference Forkability. The framework distinguishes *inference forkability* (running weights) from *training forkability* (reproducing the model). Only training forkability satisfies FORK for governance purposes. The evidence requirements in Table 3 are achievable: published training code, data manifests, and compute cost estimates.

The claim is not that all AI fails FORK, but that systems *withholding these artifacts* fail. OLMo, Pythia, and academic models demonstrate that FORK-compliant AI is possible. The question is whether frontier labs *choose* to comply.

Furthermore, the framework implies that **compute is itself DPI**. If legal forkability is insufficient without practical compute accessibility, then public compute infrastructure becomes a governance requirement—not a reason to abandon the standard.

10 Discussion

10.1 Implications for AI Governance

This framework offers three contributions to AI governance discourse:

1. **Structural vs. Behavioral Focus.** Most AI governance frameworks focus on behavioral outcomes (bias, accuracy, harm). This framework focuses on structural preconditions: can affected parties correct the system regardless of what harms emerge? Structural corrigibility is necessary for any behavioral governance to function.
2. **Distinguishing Open-washing.** The LWD-R requirement and compute capture analysis provide falsifiable tests for “open AI” claims. Systems publishing weights without training data, code, or accessible compute achieve only inference forkability—performative transparency without correction capacity.
3. **Agentic Economy Readiness.** As autonomous agents proliferate, infrastructure must become machine-readable. The five tests, reinterpreted as API contracts, specify the minimum requirements for agentic interoperability. Incurable infrastructure blocks the agentic economy.

10.2 Limitations

1. **Threshold calibration is underdetermined.** The compute capture threshold κ , WCC trigger values, and RCR operationalization require empirical calibration that this paper does not provide.
2. **Rapid technological change.** AI architectures evolve faster than governance frameworks. The risk profiles in Table 4 may become obsolete as new architectures emerge.
3. **LWD-R is aspirational.** No current frontier model satisfies full LWD-R disclosure. The requirement describes what *should* exist for corrigibility, not what currently does.
4. **Compute accessibility is geopolitically constrained.** Public compute infrastructure sufficient for training forkability may be infeasible for many nations due to cost, expertise, and supply chain constraints.
5. **The framework does not address emergent capabilities.** Unpredictable capabilities that emerge during training present governance challenges that structural tests alone cannot address.

10.3 Future Work

1. **Empirical RCR measurement:** Developing quantitative methods to assess ontological contestability
2. **Compute accessibility indices:** Cross-national comparison of training forkability capacity
3. **GDoS simulation:** Modeling governance failure under agentic scaling
4. **LWD-R schema standardization:** Machine-readable formats for disclosure requirements
5. **Regulatory integration:** Mapping framework tests to emerging AI regulations (EU AI Act, etc.)

11 Conclusion

The five corrigibility tests—EXIT, CODE, AUDIT, GOVERN, FORK—remain invariant for learned systems. What changes is the verification method and the resource barriers to compliance. This paper extends the framework through three contributions: (1) LWD-R disclosure requirements that extend CODE to training pipelines, (2) the Temporal Variety Gap explaining why AI corrigibility requires ongoing governance, and (3) Compute Capture as a resource barrier that transforms “open weights” into performative transparency.

The analysis reveals that structural corrigibility is distinct from AI alignment. Alignment asks whether operators can control systems; corrigibility asks whether affected subjects can correct them. As AI transitions from research artifact to public infrastructure, this distinction becomes essential. Infrastructure that affects citizens at population scale must be correctable by those it governs—regardless of whether that infrastructure executes deterministic code or learned parameters.

Acknowledgments

This paper extends the DPI corrigibility framework [Author, 2026] to learned systems. The author thanks the AI governance research community for ongoing critique.

License

This work is released under CC0 1.0 Universal (Public Domain).

References

- Ashby, W. Ross. *An Introduction to Cybernetics*. Chapman & Hall, 1956.
- Gebru, Timnit, et al. Datasheets for Datasets. *Communications of the ACM*, 64(12):86–92, 2021.
- Mitchell, Margaret, et al. Model Cards for Model Reporting. *Proceedings of FAT* 2019*, 220–229, 2019.
- [Author]. Corrigibility as a First Principle for Digital Public Infrastructure. *Working Paper*, 2026. Companion paper establishing the five-test framework for DPI.